

STU22004 - Applied Probability I

Group Project

Oliver Easterbrook, Finn Sprague, Lucia Brown

Introduction

Question 1

In this situation, we introduce random variable $X(t)$ representing the temperature, which varies randomly over time. As we are assuming the change in temperature over time to follow a normal distribution with mean of 0 and variance of Δt , we can say that $\frac{dX(t)}{dt}$ follows distribution $N(0, \Delta t)$; we will represent the variable $\frac{dX(t)}{dt}$ as $\Delta X(t)$. Hence, our temperature $X(t)$ can be calculated at each interval by adding $\Delta X(t)$ to $X(t)$, starting at $X(0) = 0$.

- Whenever $X(t) > 0$, we will add Δt to variable P until $t = 1$, at which point P will equal the proportion of time at which $X(t) > 0$.
- Storing the highest observed value of $X(t)$ as $Xmax$, we will set variable $Tmax$ to current time T every time a new $Xmax$ is observed. When $t = 1$, $Tmax$ will equal the time for which the temperature was at it's maximum.

There are many methods to generating a normal random variable, Box-Muller, Ahrens-Dieter, but out of all the options, I decided to use the Marsaglia-Bray method due to it reproducing more accurate results for both mean and standard deviation than Box-Muller for both small and large sample sizes:

Marsaglia-Bray Method

1. Generate two random numbers A and B from $U(0,1)$
2. Set $V_1 = (2A - 1)$, $V_2 = (2B - 1)$ and $S = V_1^2 + V_2^2$
3. Start from Step 1 if $S > 1$
4. Return the standard normal variables $Z_1 = \sqrt{-\frac{2 \log \log(S)}{S}} V_1$ and $Z_2 = \sqrt{-\frac{2 \log \log(S)}{S}} V_2$

I implemented this method using the below R code, changing my dt and repetitions variable to account for time changes of different lengths:

Function	Description
getData()	Calls functions getZs and getXs to return an array of length runs+1 of integers representing temperature $X(t)$ at time T .
getPs()	Traverses array of temperatures X and increments a variable by dt if $X[i] > 0$, before returning the variable.

```

repetitions = 1000
dt = 0.0001
t = seq(0, 1, dt)
runs = 1/dt
P <- rep(0, repetitions)
Tmax <- rep(0, repetitions)
for(i in 1:repetitions){
  X <- getData(runs)
  Tmax[i] <- getTmax(X)
  P[i] <- getPs(X)
}
avgP <- mean(P)
avgTmax <- mean(Tmax)

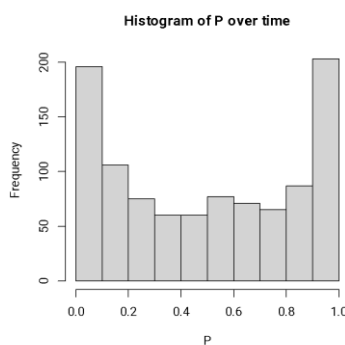
```

getTmax()	Instantiates a $T_{\max}$, $X_{\max}$ and T as 0, then traverses the array of temperatures. Every time $X[i] > X_{\max}$, $X_{\max}$ becomes $X[i]$ and T increments by dt . If $X[i] \leq X_{\max}$, then if $T > T_{\max}$, $T_{\max}$ becomes T and T becomes 0.
---------------	---

Below is the table of avgP, representing the average value of P under varying numbers of repetitions from 100 to 10,00 and different time intervals, from 0.01 to 0.0001 seconds. As we can see, the values become less varied as the number of repetitions increase, making them more accurate and reliable, whilst the values tend towards ~ 0.4999 as the time intervals decrease, suggesting that $E(P) = 0.4999$:

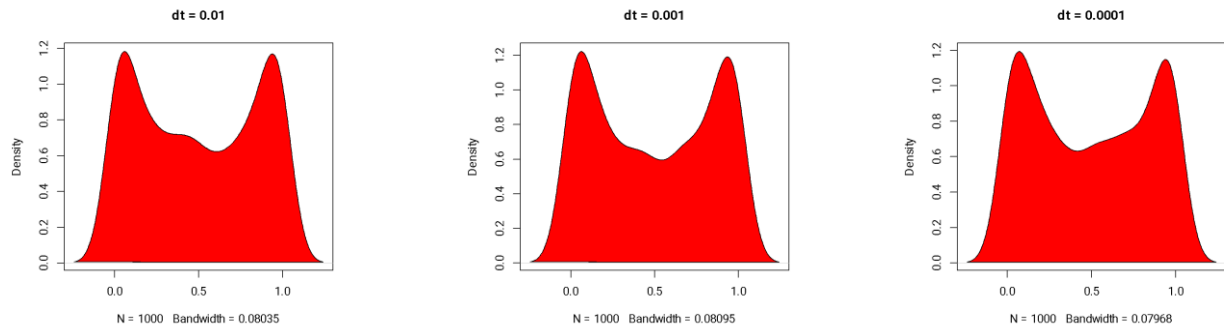
Repetitions/Dt(s)	0.01	0.001	0.0001
100	0.47060	0.50133	0.47204
1,000	0.49614	0.49895	0.49875
10,000	0.50155	0.50026	0.49945

Below is the histogram for 1,000 repetitions at a dt of 0.001s, which should allow us to observe any obvious distribution trends:



This result is unexpected. As $P(\Delta X(t) > 0) = \sim 0.5$, it would be expected that $X(t)$ would hover around 0 and hence P would be distributed approximately normally around $P = 0.5$. Instead, this graph suggests that if $\Delta X(t) > 0$ more than once in a row, then the chance of it dropping below zero again is too low to consistently achieve and vice versa, hence these strange results.

To look at this distribution in more detail, I altered the code to plot the density at each value of P to get a better grasp of it. Here is the graph for the density of each value of P for 1000 repetitions:

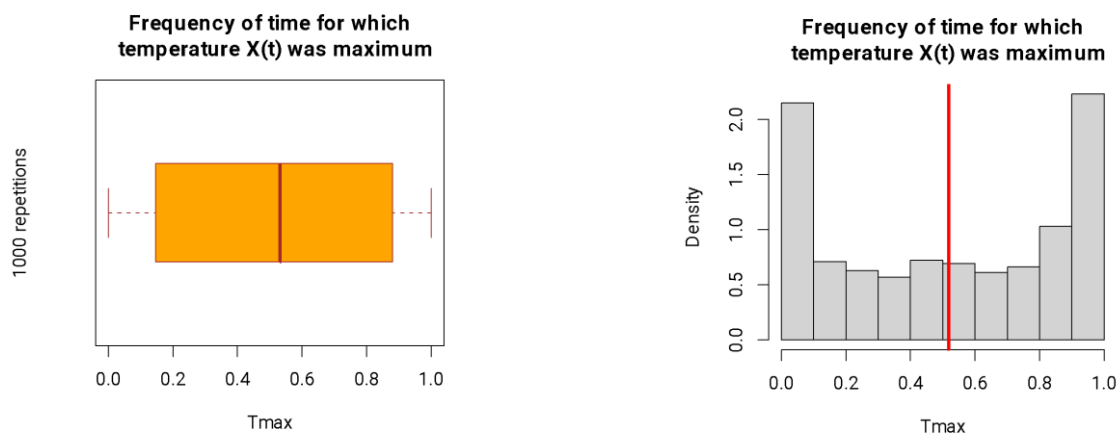


Due to the consistency between graphs, we can observe that the data is distributed with a symmetric bimodal distribution, with two peaks at opposing ends of the graph, notably towards both 0 and 1, suggesting that once the temperature is above or below 0, it is hard to make it return to a lower level. As these peaks are distributed to such extremes, it is accurate to say that $\sigma^2(P)$ and $Var(P)$ are very large.

Below is the table of avgTmax following the same structure as avgP. We can observe that as our number of repetitions increased, the value approached a tight bound of 0.495 to 0.504, suggesting that $E(T_{max}) \approx 0.5$. Furthermore, as our dt decreased, we could see that the values had a lower distribution, suggesting that the $Var(T_{max})$ is very low as dt approaches 0.

Repetitions/Dt	0.01	0.001	0.0001
100	0.54540	0.55279	0.53148
1,000	0.48121	0.51748	0.48905
10,000	0.49483	0.50387	0.49978

I decided to plot both a density histogram and a box plot of the data for 1,000 repetitions when dt = 0.001:



We can see from our boxplot that, surprisingly, the spread of the values is so large that there are no outliers present in our data; the quartiles are also in the extreme 20% of the graph as opposed to a

uniform distribution in which case they are expected to be ~25%. Whilst our mean is ~0.5, this graph shows us that the data is most likely bimodal around the extreme values, although it isn't clear. The histogram shows this much clearer, proving that the data is highly bimodal, following extremely similar trends to that of P .

Bimodal data cannot be modelled by known distributions such as Normal or Poisson, yet usually it suggests two separate outcomes occurring within a test. In this case, we have two outcome results:

1. $X(t)$ reaches it's maximum values very early - $< 0.1s$
2. $X(t)$ reaches it's maximum values very late - $> 0.9s$

This is probably due to either the temperature dipping below 0 very early and struggling to return to 0 and vice versa: This agrees with our observed distribution of P , where temperature $X(t)$ became either positive or negative early on and rarely changed back!

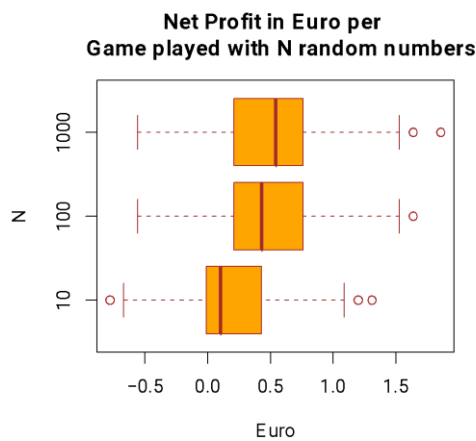
Question 2

In this question, first we must analyze the game within the question. We can simulate this game using two variables, p and q , where p represents the chance of success and q represents the chance of failure. As these are the only two possibilities of this game, $q = (1 - p)$. Therefore, our game has an expected return of $E(X) = (p)(10) - (1 - p)(1) = 11p - 1$. Ideally, we want to gain money, hence $E(X) > 0$, therefore $11p - 1 > 0 \rightarrow p > \frac{1}{11}$ gives us a profit.

We need to simulate the creation of N uniform random variables, then compare each of them to the limits $\frac{1}{N} < N[i] < \left(1 - \frac{1}{N}\right)$. The generation can be done easily in R with the built-in function 'runif', yet the game had to be created as a function in R, as done below:

```
wingame <- function(X, n){
  win = TRUE
  for(i in 1:length(X)){
    if(X[i] < 1/n || X[i] > 1 - 1/n) win = FALSE
  }
  win
}
```

I decided that to gain a useful array of data, for each value of $N = \{10, 100, 1000\}$, I would repeat the game 100 times and store the mean probability of a win (p) in a vector, then repeat *that* process within a Monte Carlo situation as to develop a reliable and vast data set. After doing so, I used the previous formula of $E(X) = (p)(10) - (1 - p)(1)$ to formulate the net profit using each value of N and plotted it in a box plot.



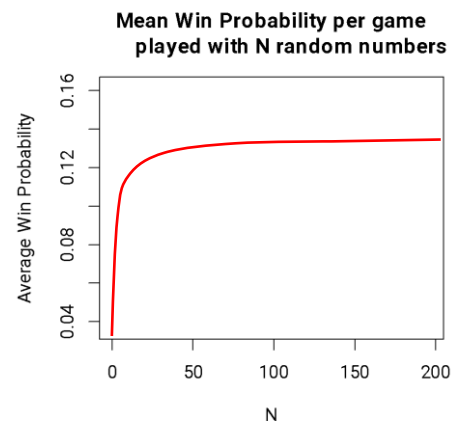
We can identify from the plot that the larger that N becomes, the higher profit is gained, as can be seen from comparing the value of the mean from each value of N .

For $N = 10$ in particular, the mean profit is just barely above €0, with the lower quartile being negative, hence it is easily the worst choice for this game. Strangely, $N = 100$ and $N = 1,000$ have nearly identical Quartile Ranges and Extremes. The change observed between $N = 10$ and $N = 100$ was significant in all areas of the plot, suggesting that the relationship between Net Profit and N is **not linear**.

Theoretically, the higher N becomes, the larger the profit becomes, hence $N = 1,000$ is the most profitable, as can be seen with its higher expected payout than for $N = 100$. However, due to the possibility of the relationship being logarithmic, the difference between playing with numbers $N = 1,000$ and $N = 10,000$ may be insignificant.

I altered code to run the game with $N = \{3, 4, 5, \dots, 200\}$ to observe the trend in the average win probability and got the following scatter plot.

As suspected, the data is spread such that a logarithmic line could easily be drawn through the points to act as a line of best fit, hence the profit of this game will most likely plateau very close to that of $N = 1000$, for which we gain ~0.5€. Unfortunately, generating many numbers would take a while. Would not recommend to those in need of money fast.



By consulting the above *papersTable* Excel sheet, the maximum paper could be located in any position. There is no indication that the median is more likely to contain the maximum paper. This gives the median paper approach the same probability as the first paper approach: $1/n$

This is proven by the Python code developed, which will return L most of the time, especially for large permutations of n

```

1 import statistics
2 import random
3 import numpy as np
4 #strategy 5 : always choose the median card
5
6 numberOfPapers = 1000
7 listIndex = []
8 i = 1
9 while(i <= numberOfPapers):
10     listIndex.append(i)
11     i += 1
12
13 listOfPapers = np.random.permutation(100000)[:numberOfPapers]
14 listOfPapers = list(listOfPapers)
15 maxPaper = max(listOfPapers)
16 #print("List of Papers : ", listOfPapers)
17
18 mid = len(listIndex) // 2
19 res = (listIndex[mid] + listIndex[~mid]) / 2
20 if(res != int):
21     res = res + 0.5
22     res = int(res)
23
24 medianElement = listOfPapers[res-1]
25 if(medianElement == maxPaper):
26     print("W")
27 else:
28     print("L")
29

```

Strategy 3: A strategy for choosing the paper with the highest number can be determined with the Optimal Stopping Problem, also known as the Secretary Problem.

Given the n papers, a sample must be determined, then choose the next paper which exceeds the sample. A sample is required because, as seen in strategy 1, choosing the first paper only guarantees a probability of $1/n$

If $n = 3$ and the player selects randomly, there is a $1/3$ chance of winning. If the player always skips the 1st paper, they have a sample. The player chooses the next paper than is higher than paper 1. This increases the chance of winning to $1/2$

The solution to the Optimal Stopping Problem is the 37% rule or the $1/e$ Law of Best Choice proved by F. Thomas Bruss. Applying the 37% rule to the papers, the first $1/e$ papers are taken as the required sample.

If $n = 10$: $37\% = 3.7 \text{ papers} \approx 4 \text{ papers} = [9, 7, 4, 3, 8, 5, 1, 2, 6, 10]$ = Papers (9, 7, 4, 3) are the sample.

The highest in the sample is 9. The player would turn over papers until reaching a number higher than 9. Which is 10. 10 is the highest number in the set so the player wins.


```

1 import numpy as np
2 numberOfPapers = 100
3
4 listOfPapers = np.random.permutation(1000)[:numberOfPapers]
5 listOfPapers = list(listOfPapers)
6 maxPaper = max(listOfPapers)
7
8 thirtySevenPercent = 0.37
9 divisionOfPapers = round(numberOfPapers * thirtySevenPercent)
11 sample = []
12 for i in range(divisionOfPapers-1):
13     sample.append(listOfPapers[i])
14
15 for i in range(divisionOfPapers-1):
16     listOfPapers.remove(listOfPapers[i])
17
18 maxSample = max(sample)
19 #print("Max of Sample : ",maxSample)
21 newMax=0;
22 for i in range(len(listOfPapers)):
23     if(listOfPapers[0] > maxSample):
24         newMax = listOfPapers[0]
25         #print("First Paper Greater than ",maxSample," = ",newMax)
26         break
27     listOfPapers.remove(listOfPapers[0])
28
29 if (newMax == maxPaper):
30     print("W")
31 else:
32     print("L")

```

The below table summarises the probabilities of the three strategies discussed

Strategy	Probability
Always choose first number	1/n
Always choose median	1/n
37% Rule	0.37

Question 4

In this game, 100 cards with the numbers from 1 to 100 are shuffled and shown one by one. On turn 1, if the card turned over is a '1' then that is considered a *hit*. The same is true if the '2' card is dealt on turn 2, and '3' on turn 3, and so on. In short, every turn, the only possible outcomes are *hit* or *no hit*.

Let X be a random variable that takes 1 if the card is a *hit* and 0 if it is not. Since the cards are in random order and they are non-repeating, then the chances of getting a *hit* on any turn is $1/100$. So $E(X)$ for any turn is $1/100$. One game lasts 100 turns, so $E(X)$ for a full game is 1.

To test this, I wrote some code in Python using the matplotlib package to graph my results. I started by importing the necessary packages, and a function that simulates one turn.

```

import random
import matplotlib.pyplot as plt
import matplotlib as mpl
import numpy as np

def findHits():
    hits = 0
    seq = random.sample(range(1, 101), 100)
    for i in range(1, 100):
        if(i == seq[i]):
            hits+=1
    return hits

```

A function that graphs the expected number of hits and the variance, depending on the input.

```
def simulation(numRepetitions, sel):
    # x_axis = list(range(1, numRepetitions))
    expected = []
    variance = []
    hits = 0
    totalHits = 0
    for i in range(numRepetitions):
        hits = findHits()
        totalHits += hits
        expectedValue = totalHits / (i + 1)
        expected.append(expectedValue)

        varianceValue = expectedValue - (expectedValue ** 2)
        variance.append(abs(varianceValue))

    if(sel == 'exp' or sel == 'Expected Number'):
        return expected
    else:
        return variance
```

The function above returns a list of values that can be plotted on the y-axis and show the expected value and variance converging on a single line as the number of repetitions of the game increases.

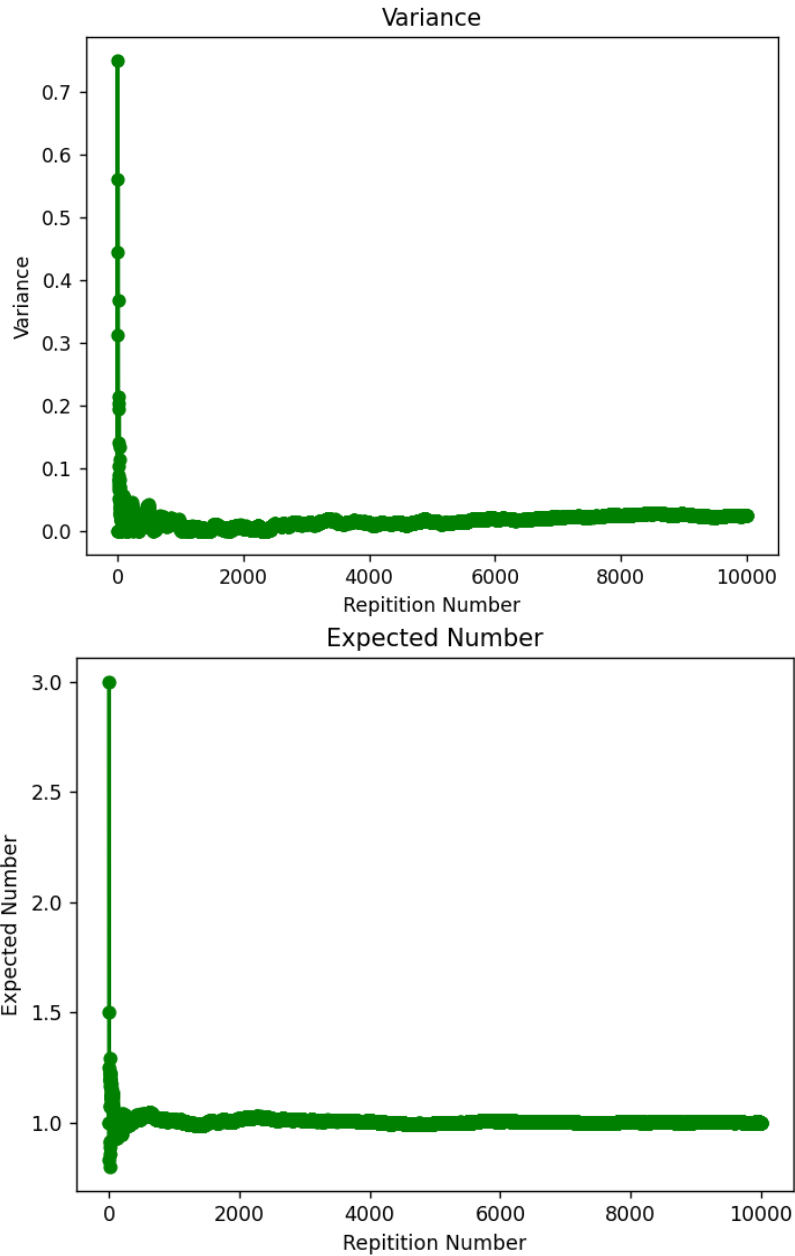
Finally, by selecting the input and calling the following function, I can create graphs displaying the expected number and variance of the random variable after 10,000 games.

```
def showGraph(numRepetitions, sel):
    x_axis = list(range(1, numRepetitions + 1))

    if(sel == 'exp' or sel == 'Expected Number'):
        y_axis = simulation(numRepetitions, sel)
    else:
        y_axis = simulation(numRepetitions, sel)

    plt.plot(x_axis, y_axis, 'go-', label='line 1', linewidth=2)
    plt.xlabel('Repetition Number')
    plt.ylabel(sel)
    plt.title(sel)
    plt.show()

showGraph(10000, 'Expected Number')
```



So, as the Expected Number begins to converge on 1, the Variance decreases, approaching 0.

This can be shown mathematically. The Variance can be written as:

$$Var(X) = E(X^2) - E(X)^2$$

And since the random variable X can only take values of 1 or 0, then $E(X^2)$ just equals $E(X)$.

Since $E(X) = 1$., as I have proven mathematically and through the graph, then $E(X)^2$ should also equal 1, meaning that the Variance of the experiment should equal 0.